

Efficienza asintotica e notazioni

Nello studio del tempo di esecuzione di un algoritmo, calcolare il numero esatto di istruzioni è inutile, perché il tempo di esecuzione dipenderebbe da hardware, compilatore e altri fattori esterni.

In questo scenario, i termini di grado inferiore e le costanti moltiplicative diventano irrilevanti, le notazioni asintotiche servono a classificare gli algoritmi in *famiglie di crescita*.

Notazione Theta Grande Θ

La notazione Theta è la più precisa delle notazioni asintotiche, dire che $f(n) = \Theta(g(n))$, o più correttamente, che $f(n) \in \Theta(g(n))$ significa affermare che le due funzioni appartengono alla stessa famiglia di crescita, cioè che crescono alla stessa velocità. Esistono due modi per dimostrare che una funzione appartiene a $\Theta(g(n))$, il metodo analitico (dei limiti) e quello formale (delle costanti)

Metodo 1

È il metodo più usato nella risoluzione degli esercizi. Due funzioni sono asintoticamente equivalenti se il loro rapporto, per n che tende all'infinito, converge ad un numero costante strettamente maggiore di zero, cioè

$$f(n) \in \Theta(g(n)) \Leftrightarrow \lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} = k > 0$$

Questo metodo funziona perché se il limite è infinito, f cresce troppo velocemente. Se il limite è zero, f cresce troppo lentamente. Se è una costante k invece, crescono (asintoticamente) alla stessa velocità.

Metodo 2

Per una data funzione f , l'insieme $\Theta(g(n))$ contiene tutte le funzioni che, da un certo punto in poi, si trovano tra due versioni "scalate" di $g(n)$.

$$\Theta(g(n)) = \left\{ f(n) \mid \exists c_1, c_2, n_0 > 0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n > n_0 \right\}$$

Con n_0 si intende semplicemente un valore per cui, da quel dato valore in poi, la regola vale fino all'infinito.

c_1 e c_2 sono i fattori di scala, dimostrano che $f(n)$ è identica a $g(n)$ se moltiplicata per una costante inferiore e una superiore

Notazione O Grande (O)

Visto che la notazione Theta richiedeva un limite stretto, la notazione grande fornisce solo un **limite asintotico superiore**. Dire che $f(n) = O(g(n))$ significa affermare che la funzione $f(n)$ non cresce più velocemente di $g(n)$. Possiamo dire, in un certo senso, che rappresenta lo scenario peggiore in un algoritmo.

Definizione tramite limite

Una funzione appartiene a $O(g(n))$ se il rapporto tra le due funzioni, all'infinito, si annulla (cioè se $g(n)$ è un infinito di ordine superiore) oppure se converge a una costante

$$f(n) = O(g(n)) \Leftrightarrow \lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} < +\infty$$

Definizione formale

L'insieme $O(g(n))$ contiene tutte le funzioni che, da un certo punto n_0 in poi, non superano mai la funzione $g(n)$ moltiplicata per una costante c .

$$O(g(n)) = \left\{ f(n) \mid \exists c, n_0 \in \mathbb{R}^+ : 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0 \right\}$$

Notazione Omega Grande Ω

La notazione Ω è l'esatto opposto di O grande. Fornisce un limite asintotico inferiore. Dire che $f(n) = \Omega(g(n))$ significa affermare che la funzione $f(n)$ non cresce più velocemente di $g(n)$. Rappresenta in genere il caso migliore di un algoritmo

Definizione tramite limite

Una funzione appartiene a $\Omega(g(n))$ se il rapporto tra le due funzioni all'infinito esplode (cioè $f(n)$ è di ordine superiore) oppure converge a una costante.

$$f(n) = \Omega(g(n)) \Leftrightarrow \lim_{n \rightarrow +\infty} \frac{f(n)}{g(n)} > 0$$

Definizione formale

L'insieme $\Omega(g(n))$ contiene tutte le funzioni che, da un certo n_0 in poi, stanno sempre al di sopra della funzione $g(n)$ moltiplicata per una costante c .

$$\Omega(g(n)) = \left\{ f(n) \mid \exists c, n_0 \in \mathbb{R}^+ : 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0 \right\}$$

